

3546. Equal Sum Grid Partition I

Solved 

Medium

 Topics

 Companies

 Hint

You are given an $m \times n$ matrix `grid` of positive integers. Your task is to determine if it is possible to make **either one horizontal or one vertical cut** on the grid such that:

- Each of the two resulting sections formed by the cut is **non-empty**.
- The sum of the elements in both sections is **equal**.

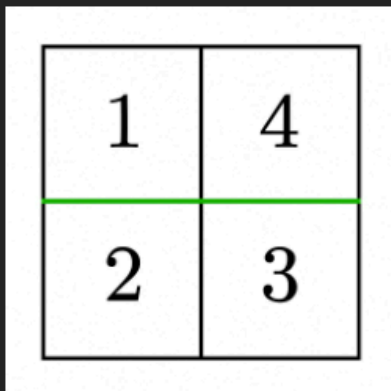
Return `true` if such a partition exists; otherwise return `false`.

Example 1:

Input: `grid = [[1,4],[2,3]]`

Output: `true`

Explanation:



1	4
2	3

A horizontal cut between row 0 and row 1 results in two non-empty sections, each with a sum of 5. Thus, the answer is `true`.

Example 2:

Input: grid = [[1,3],[2,4]]

Output: false

Explanation:

No horizontal or vertical cut results in two non-empty sections with equal sums. Thus, the answer is `false`.

Constraints:

- $1 \leq m == \text{grid.length} \leq 10^5$
- $1 \leq n == \text{grid}[i].\text{length} \leq 10^5$
- $2 \leq m * n \leq 10^5$
- $1 \leq \text{grid}[i][j] \leq 10^5$

Python:

class Solution:

```
def canPartitionGrid(self, grid: List[List[int]]) -> bool:
```

```
    m, n = len(grid), len(grid[0])
```

```
    total = sum(sum(row) for row in grid)
```

```
    if total % 2:
```

```
        return False
```

```
    target = total // 2
```

```
    s = 0
```

```
    for i in range(m - 1):
```

```
        s += sum(grid[i])
```

```
        if s == target:
```

```
            return True
```

```
    s = 0
```

```
    for j in range(n - 1):
```

```
        for i in range(m):
```

```
            s += grid[i][j]
```

```
    if s == target:
        return True
```

```
    return False
```

JavaScript:

```
/**
 * @param {number[][]} grid
 * @return {boolean}
 */
var canPartitionGrid = function(grid) {
    const m = grid.length, n = grid[0].length;
    let total = 0;

    for (let i = 0; i < m; i++)
        for (let j = 0; j < n; j++)
            total += grid[i][j];

    if (total % 2 !== 0) return false;

    const target = total / 2;
    let sum = 0;

    for (let i = 0; i < m - 1; i++) {
        for (let j = 0; j < n; j++)
            sum += grid[i][j];
        if (sum === target) return true;
    }

    sum = 0;

    for (let j = 0; j < n - 1; j++) {
        for (let i = 0; i < m; i++)
            sum += grid[i][j];
        if (sum === target) return true;
    }

    return false;
};
```

Java:

```
class Solution {
    public boolean canPartitionGrid(int[][] grid) {
        int m = grid.length, n = grid[0].length;
        long total = 0;
```

```

for (int[] row : grid)
    for (int x : row)
        total += x;

if ((total & 1) == 1) return false;

long target = total / 2, sum = 0;

for (int i = 0; i < m - 1; i++) {
    for (int j = 0; j < n; j++)
        sum += grid[i][j];
    if (sum == target) return true;
}

sum = 0;

for (int j = 0; j < n - 1; j++) {
    for (int i = 0; i < m; i++)
        sum += grid[i][j];
    if (sum == target) return true;
}

return false;
}
}

```